

Extending Family-Based Vulnerability Discovery for Highly-Configurable Software Systems

Proposal
of

**Fynn Tschacher
Chaoyang Wei**

at the Department of Informatics
Institute of Information Security and Dependability
Test, Validation and Analysis (TVA)

Supervisor:

Tim Bächle

Completion period: 18.05.2026 – 30.09.2026

Contents

1 Introduction 3

1.1 Problem 3

1.2 Goal 4

2 Work Packages 4

3 Division of Work 8

4 Implemented Work Packages 9

5 Current State 9

1 Introduction

In a world where software has an ever-growing impact on our everyday lives due to the ongoing digitalization and has become a central component of modern communication, mobility, critical infrastructure, healthcare systems, and financial systems, its correctness and security are of central importance to our society. At the same time, the complexity of modern software systems is constantly increasing. Today, software often no longer consists of individual, isolated programs but rather of numerous complex, configurable components that form software families or software product lines [1]. This increasing complexity makes it difficult to detect errors and vulnerabilities early. Security vulnerabilities in central software components and libraries over the past few years have demonstrated just how serious they can be. A famous example is the Heartbleed vulnerability (CVE-2014-0160), discovered in April 2014 in OpenSSL. This vulnerability was contained in the Heartbeat extension, an optional OpenSSL extension, that was included by default at compile time via compiler flags. It enabled attackers to read sensitive memory contents from a remote system [4, 5]. Another well-known vulnerability is Log4Shell (CVE-2021-44228). A vulnerability discovered in November 2021 in the widely used Java library Log4j, which, under certain conditions, allowed remote code execution. The far-reaching consequences that errors and manipulations in today’s software systems can have were made clear by the CrowdStrike incident in July 2024. As a faulty software update from the vendor CrowdStrike caused Windows systems worldwide to crash [2, 6]. Although this was not an exploited vulnerability but rather a software bug, the incident illustrates the consequences that faulty or malicious software can have today. Thus, this incident highlights the importance of systematically investigating software and software families for errors and security vulnerabilities.

1.1 Problem

Existing static vulnerability analysis techniques are designed for classical, non-configurable software. As a result, they are not capable of supporting configurable software systems where variability is introduced via conditional compilation (e.g., `#ifdef`). In such systems, configuration options create complex dependencies between code fragments. According to Thüm et al. [5], vulnerabilities in software families may only manifest under specific configuration combinations or arise from interactions between features, that maybe be configurable, making them nearly impossible to detect using traditional methods.

Current industry practices typically rely on a “product-based” analysis strategy, where individual configuration instances of a configurable software system/software families are generated and analyzed independently. However, as the configuration space grows exponentially, this approach faces severe scalability limitations. Furthermore, because different configurations share a significant amount of common code, product-based analysis leads to massive computational redundancy [3].

More critically, existing query-based tools like JOERN are variability-oblivious. Their parsers and intermediate representations (IR) are designed for fully preprocessed source code, meaning they cannot handle configuration-related semantics of analysis. For instance, structural variations in the Abstract Syntax Tree (AST), such as configuration related function calls and control flow, cannot be represented.

This information loss prevents security researchers from performing comprehensive vulnerability discovery across the entire software family, leaving rare but critical configurations unexamined.

1.2 Goal

This project aims to extend and refine the family-based static analysis approach initiated by Dormann [3] to enable efficient and precise vulnerability discovery in highly configurable C code. While existing "lifting-by-simulation" methods have attempted to address variability, they often struggle to handle many configuration options, limiting their applicability to large-scale software families.

To overcome these challenges, our core objective is to implement a "lifting-by-extension" scheme. This involves extending Dormann's new C parser/JOERN C frontend (based on SuperC) and the Intermediate Representation (IR) of the JOERN platform. Following the technical approach in Dormann's work, we will focus on the translation of Variability-aware ASTs (VAST) from the SuperC format into the JOERN format and extending the JOERN graph representation so it can also describe configurable code.

By further extending the JOERN fork initiated by Dormann, this project will resolve existing compatibility issues and enhance the tool's ability to perform variability-aware code analysis. Ultimately, our goal is to provide a unified framework that allows security researchers to identify vulnerabilities across all potential configurations of a software families within a single analysis pass.

2 Work Packages

For the extension of JOERN to enable the analysis of variable C code, as described in Mats Dormann's thesis, a new C JOERN frontend must be implemented, and the generation algorithms for CFG (Control Flow Graph), PDG (Program Dependence Graph), and CPG (Code Property Graph) must be extended [3].

All tasks required for this extension are listed in the [table below](#). The "Status" column of the table shows the current implementation status of each task in Dormann's implementation. The symbol  indicates that the task has not yet been started or the implementation is not functional,  indicates that it has been started but is incomplete, and  indicates that the task is already completed. In addition, some tasks are marked with "[optional]". These tasks will be treated with a lower priority during the project, as they are not strictly necessary for the basic support of variable C code.

Task	Description	Status
1	unary operators Implement the support for the 10 missing unary operators: <code>"++<var>", "--<var>", "+<var>", "-<exp>", "*<exp> "&<var>", "~<log-exp>", "!<log-exp>", "sizeof(<var>)", "typeof(<var>)" and "<exp>". The operators "<var>++" and "<var>--" are already supported.</code>	✓
2	binary operators Implement the support for the missing binary operators: <code>"<<", ">>", "&", "^", " ", "&&", " ", "<var> *=", "<var> /=", "<var> %=", "<var> +=", "<var> -=", "<var> <=", "<var> >=", "<var> &=", "<var> ^=", "<var> =", "!=", "<var>.<arg>", "<var>-><arg>", "OperatorMax", "OperatorMin" and "<type> <var>". The following operators are already supported: <code>"*", "/", "%", "+", "-", "<var> =", ">", "<", "<=", "==" and ">="</code></code>	✓
3	variable declaration Implement the conversion of single variable declarations, e.g. <code>"int x = 4;"</code>, from the SuperC-VAST format to the JOERN-VAST format.	✓
4	multi inline declaration Implement the conversion of multiple inline variable declarations, e.g. <code>"int x = y = 5;"</code> and <code>"int x = 4, y = 5;"</code>, from the SuperC-VAST format to the JOERN-VAST format. It may be necessary to handle the code fragment reconstruction from the SuperC-VAST representation for multiple inline variable declarations separately in the <code>code(...)</code> method.	✗
5	typedef and struct Implement the conversion of <code>typedef</code> and <code>struct</code> declarations from the SuperC-VAST format to the JOERN-VAST format. The processing of <code>typedef</code> and <code>struct</code> during the generation of the CFG, PDG, and CPD should, if possible and not disrupted by conditional macro, be handled by the standard JOERN implementation.	✗
6	assembly code and register access [optional] Implement the conversion of Assembly code and register access from the SuperC-VAST format to the JOERN-VAST format. The processing of the Assembly code and the register access during the generation of the CFG, PDG, and CPD should, if possible and not disrupted by conditional macro, be handled by the standard JOERN implementation.	✗
7	enum Complete the <code>enum</code> conversion implementation from the SuperC-VAST format to the JOERN-VAST format. The processing of <code>enum</code> during the generation of the CFG, PDG, and CPD should, if possible, be handled by the standard JOERN implementation.	✓

Task	Description	Status
8	<code>goto and goto labels</code> [optional] Implement the support for <code>goto</code> statements with conditional-macro-dependent labels. This implementation has to extend not only to the creation of the JOERN-VAST, but also the JOERN generators for the VCFG, VPDG, and VCPG.	✗
9	<code>multi file support</code> [optional] Implement the support for analyzing a software project that consists of multiple files.	✗
10	<code>multi function handling</code> Implement the support for multiple function declarations that are not modifiable by conditional macros. The processing of these function during the generation of the CFG, PDG, and CPD should be handled by the standard JOERN implementation.	●
11	<code>code(...)</code> Complete the implementation of the code generation method " <code>code(...)</code> ". The current implementation returns in some cases no, too much or incomplete code string for a given VAST subtree. In addition, in some cases it has trouble separating function arguments with commas, separating expressions and statements with semicolons, and placing parentheses (" <code>()</code> ", " <code>{}</code> "). This method also has to handle macro definitions and conditional macros.	●
12	<code>if statements</code> Implement the conversion of the <code>if</code> , <code>else if</code> and <code>else</code> statement from the SuperC-VAST format to the JOERN-VAST format. The processing of <code>if</code> , <code>else if</code> and <code>else</code> during the generation of the CFG, PDG, and CPD should, if possible, be handled by the standard JOERN implementation.	●
13	<code>switch-case statement</code> Implement the conversion of the <code>switch-case</code> statement from the SuperC-VAST format to the JOERN-VAST format. The processing of <code>switch-case</code> during the generation of the CFG, PDG, and CPD should, if possible, be handled by the standard JOERN implementation.	✗
14	<code>code block</code> Complete the implementation of the code block generation, as in some cases these blocks do not exist when expected or contain an incomplete reconstructed code fragment of the corresponding VAST subtree. (The second issue may be related to the incomplete implementation of the " <code>code(...)</code> " function.)	●
15	<code>function declaration</code> Implement the conversion of a function declaration with parameters from the SuperC-VAST format to the JOERN-VAST format. The processing of function declarations during the generation of the CFG, PDG, and CPD should, if possible, be handled by the standard JOERN implementation.	✓

Task	Description	Status
16	function call Implement the conversion of a function call with parameters from the SuperC-VAST format to the JOERN-VAST format. The processing of function calls during the generation of the CFG, PDG, and CPD should, if possible, be handled by the standard JOERN implementation.	✓
17	static assertions [optional] Implement the conversion of static assertions (<code>_Static_assert(...)</code> and <code>static_assert(...)</code>) from the SuperC-VAST format to the JOERN-VAST format. The processing of static assertions during the generation of the CFG, PDG, and CPD should, if possible and not disrupted by conditional macro, be handled by the standard JOERN implementation.	✗
18	dot-graph generator Extend the two dot-graph generator methods for JOERN-VAST so that they can visualize all patterns of a VAST.	●
19	while loop and do-while loop Implement the conversion of the <code>while(...){...}</code> loop and the <code>do{...}while(...);</code> loop from the SuperC-VAST format to the JOERN-VAST format. The processing of <code>while(...){...}</code> loops and <code>do{...}while(...);</code> loops during the generation of the CFG, PDG, and CPD should, if possible, be handled by the standard JOERN implementation.	✗
20	for loop Implement the conversion of the <code>for(..., ..., ...){...}</code> loop from the SuperC-VAST format to the JOERN-VAST format. The processing of <code>for(..., ..., ...){...}</code> loops during the generation of the CFG, PDG, and CPD should, if possible, be handled by the standard JOERN implementation.	✗
21	break and continue instruction Implement the conversion of the <code>break</code> and <code>continue</code> instruction from the SuperC-VAST format to the JOERN-VAST format. The processing of <code>break</code> and <code>continue</code> instructions during the generation of the CFG, PDG, and CPD should, if possible, be handled by the standard JOERN implementation.	✗
22	simple parameterized macro Implement the conversion of simple macros with parameters from the SuperC-VAST format to the JOERN-VAST format.	●
23	conditional macro Implement the conversion of a condition macro from the SuperC-VAST format to the JOERN-VAST format. The implementation should allow conditional macros to manipulate or modify code blocks, method signatures, variable declarations, variable types, array lengths, enums, and custom data types/structs.	●

3 Division of Work

Regarding the division of work, we will implement a dynamic task allocation model based on the task pool. The work packages and implementation statuses outlined in [Section 2](#) will be treated as an open backlog, managed similarly to GitHub issues. Team members will independently select and claim pending tasks from this pool based on the current project phase. A task is considered as complete (✓) once its functionality is implemented. Since the workload and complexity of the tasks outlined in [Section 2](#) are difficult to estimate accurately at this early stage, a fixed assignment could lead to an uneven distribution of responsibilities. Consequently, team members will flexibly select tasks based on project progress and ongoing internal communication. A detailed task assignment list will be progressively refined and updated as the development work unfolds.

4 Implemented Work Packages

Overview of the extend to which the work packages have been implemented and what issues have been encountered/remain.

Task	Description	Assignees	Status
1	unary operations handling	Chaoyang	✓
2	binary operations handling	Chaoyang	✓
3 & 4	multi variable declaration	Fynn	✓
5	<i>typedef</i> and <i>struct</i> declaration		✗
6	assembly code and register access handling [optional]		✗
7	<i>enum</i> declaration	Chaoyang	✓
8	<i>goto</i> and <i>goto label</i> handling	Chaoyang	✓
9	multi file support [optional]		✗
10 & 15	multi function declaration	Fynn	●
11	code generation/reconstruction	Chaoyang , Fynn	✓
12	<i>if</i> statements	Chaoyang	✓
13	<i>switch</i> -case statements	Chaoyang	✓
14	code block handling	Fynn	✓
16	function call handling	Chaoyang	✓
17	static assertions [optional]		✗
18	dot-graph generation for JOERN and SuperC	Fynn	✓
19	<i>while</i> and <i>do-while</i> loop handling	Chaoyang	✓
20	<i>for</i> loop handling	Chaoyang	✓
21	<i>break</i> , <i>continue</i> instruction handling	Chaoyang	✓
22	simple parameterized macro handling		✗
23	conditional macros handling	Fynn	✓

5 Current State

The implementation from Dormann has been migrated to the new command- and feature-handler-based SuperC-to-JOERN AST translation. His implementation still exists in the project but is no longer used for the translation. The translation entry point is located in the `VAstCreatorNew`. The current implementation is capable of translating all C features marked as complete (✓) in the table in Section 4 with full conditional support. None of the tasks marked with ✗ have been implemented. Tasks 10 and 15 have been started and are mostly implemented. However, the

conditional handling for function declarations is nearly complete. But support for conditional return-type pointer degree and the conditional decision of whether an array is returned is missing. So all other aspects of a function declaration can already be translated, even if they are conditional. This also applies to multi-method support (Task 10).

General notes on the implementation:

1. Each C feature has its own `VastPatternConverter`, which implements `io.joern.c2cpg.astcreation.converter.VastPatternConverter` and must be registered in `VASTConverterForC`.
2. Functionality required by multiple C features is outsourced to so-called `VastFeatureHandlers`, which also has to be registered in `VASTConverterForC`.
3. The location information methods in `VASTCreatorNew` should not be used, because it is simpler to extract the code positions directly from the SuperC AST (from the Language and Text nodes).
4. JOERN Choice nodes can now have more than 2 sub-ASTs and never have a code position specified.
5. JOERN return nodes have a code position specification only if they have a return argument; otherwise, they have no code position specification.
6. The JOERN dot-graph plotting functionality now offers options for coloring important nodes, outputting important node parameters, or edge-type coloring.
7. The SuperC AST can now also be output as a dot graph.
8. Red colored JOERN nodes in the VAST or other JOERN graphs indicate an incomplete translation caused by missing C feature support.
9. All tests are located in the package: `io.joern.c2cpg.viability.vast`

Notes on remaining tasks and adjustments to the other graph creators (CFG, ...):

1. It is recommended to implement `typedef` and `struct` declarations as `VastFeatureHandler` and, similar to the `VastPatternConverterForConditionalMacro`, to define an additional `VASTPatternConverter` that internally uses the `VastFeatureHandler`, because `typedef` and `struct` declarations can be combined with variable declarations, meaning that a `typedef` or `struct` declaration can be a sub-AST of a variable declaration or a multi-variable declaration.
2. It is essential to ensure that control flows and data dependencies are handled correctly for complex features such as method declarations or switch cases, especially since a major adjustment has been made in the modeling of conditional return types for method declarations and `Choice` nodes can now have more than two children.

3. It is recommended to adjust the function parameter declaration again and use a structure similar to that of variable accesses (JOERN `Identifier` nodes) if the parameters are conditional.
4. The conditional handling of the conditional return-type pointer degree and the conditional decision of whether an array is returned is still missing and needs to be added.
5. The handling of array initializations needs to be added.

References

- [1] Sven Apel, Don Batory, Christian Kästner, and Gunter Saake. *Feature-Oriented Software Product Lines: Concepts and Implementation*. Springer Berlin Heidelberg, 1 edition, 2013.
- [2] CrowdStrike. External technical root cause analysis — channel file 291. <https://www.crowdstrike.com/wp-content/uploads/2024/08/Channel-File-291-Incident-Root-Cause-Analysis-08.06.2024.pdf>, August 2024.
- [3] Mats Oskar Dormann. Family-based vulnerability discovery for software product lines using variability-aware code property graphs. 2026.
- [4] Gabriel Ferreira, Momin Malik, Christian Kästner, Jürgen Pfeffer, and Sven Apel. Do `#ifdefs` influence the occurrence of vulnerabilities? an empirical study of the linux kernel. In *Proceedings of the 20th International Systems and Software Product Line Conference, SPLC '16*, page 65–73, New York, NY, USA, 2016. Association for Computing Machinery.
- [5] Alexander von Rhein, Thomas Thüm, Ina Schaefer, Jörg Liebig, and Sven Apel. Variability encoding: From compile-time to load-time variability. *Journal of Logical and Algebraic Methods in Programming*, 85(1, Part 2):125–145, 2016. Formal Methods for Software Product Line Engineering.
- [6] David Weston. Windows security best practices for integrating and managing security tools. <https://www.microsoft.com/en-us/security/blog/2024/07/27/windows-security-best-practices-for-integrating-and-managing-security-tools/>.